# Outsourcing Arithmetic: Stateful Sandboxes and Entropy-Weighted Consensus for Olympiad Math

**Arturo Emmanuel Díaz Domínguez** *
**Alan Daleth Hernández Barreto** *
**Karol Yenaro Morales Escobar** *

* *Instituto Tecnológico de Zacatepec, TecNM, Morelos, Mexico (e-mail: {arturo.dd,l23090959, 23090998}@zacatepec.tecnm.mx).*

**Abstract:** Mathematical reasoning in Large Language Models (LLMs) suffers from hallucination and logical inconsistency, particularly when scaling to Olympiad-level complexity. An agentic framework is presented, integrating stateful Python execution with an inverse-entropy decoding strategy to mitigate these vulnerabilities. By offloading computational overhead to a persistent Jupyter kernel, the system eliminates arithmetic degradation and facilitates autonomous error recovery. To resolve candidate consensus during inference-time scaling, a selection mechanism weighted by the inverse of the mean token-level Shannon entropy—derived from LLM logprobs—is introduced. Evaluated on the AI Mathematical Olympiad (AIMO) 3 dataset, the approach achieved 39 points under strict compute constraints. Empirical analysis demonstrates that entropy-guided majority voting significantly outperforms traditional self-consistency, providing a robust, statistically grounded architecture for verifiable neuro-symbolic reasoning.

*Keywords:* Large Language Models, Mathematical Reasoning, Entropy, Inference Optimization, Neuro-symbolic AI, Stateful Execution, Test-Time Compute.

## 1. INTRODUCTION

Automated mathematical reasoning has historically been bifurcated into two distinct paradigms: formal symbolic theorem proving and informal natural language generation. While formal systems, such as Lean and Coq, offer rigorous verification mechanisms, they require extensive human labor to translate informal problems into strict formal syntax. Conversely, the advent of Large Language Models (LLMs) has shifted the focus toward neural-driven generation, enabling systems to parse and solve complex mathematical concepts expressed in natural language (DeepSeek-AI, 2025; Trinh et al., 2024).

Despite these architectural advancements, standard auto-regressive generation struggles profoundly with Olympiad-level mathematical problems. Extended inference trajectories—often required for intricate algebraic manipulation or multi-step combinatorial logic—expose LLMs to cascading failure modes. Once a single arithmetic error or logical hallucination is introduced into the context window, the auto-regressive nature of the model amplifies the mistake, leading to an inevitable logical collapse. To mitigate this computational degradation, contemporary research has focused on decoupling logic formulation from raw numerical calculation. By integrating external, deterministic environments, such as Python interpreters, frameworks like Program of Thoughts (PoT) (Chen et al., 2023) and tool-integrated agents (Gou et al., 2024) allow the neural network to act as a logic planner while delegating execution to a rigorous symbolic engine.

Furthermore, recent literature dictates that scaling inference-time computation—often referred to as Test-Time Compute—can yield performance improvements comparable to scaling the underlying model parameters by an order of magnitude (Brown et al., 2024). Techniques such as Self-Consistency (Wang et al., 2023) have proven highly effective in stabilizing outputs by generating multiple reasoning paths and selecting the final answer through majority voting. However, traditional majority voting paradigms operate under a uniform distribution assumption; they treat all sampled responses equally, effectively ignoring the underlying uncertainty inherent in the generation process. If a model generates ten incorrect answers with high uncertainty and five correct answers with absolute statistical confidence, standard majority voting will incorrectly select the hallucinated consensus.

Recent empirical evaluations demonstrate that test-time inference can be drastically optimized by assessing the semantic uncertainty and token-level entropy of the generated sequences (Kuhn et al., 2024). Token probability distributions offer a direct window into the model's internal confidence state.

In this paper, a robust inference architecture specifically designed for the AI Mathematical Olympiad (AIMO) 3 benchmark is proposed. The framework integrates a persistent, stateful Jupyter sandbox to preserve execution context across multi-turn conversational interactions, paired with a dynamic, uncertainty-aware decoding strategy. Rather than relying on naive majority voting, an Inverse-

Entropy Voting mechanism is introduced. This mechanism aggregates consensus by weighting candidate answers proportionally to their mathematical certainty, which is approximated via the Shannon entropy of the output log-probabilities. Under strict computational boundaries and hardware limitations, the methodology achieved 39 points in the AIMO 3 competition, validating that entropy-aware decoding paired with deterministic programmatic verification provides a scalable solution to the hallucination bottlenecks present in Olympiad-level AI.

## 2. BACKGROUND AND RELATED WORK

The intersection of neural networks and symbolic computation represents a rapidly evolving domain. To contextualize the proposed architecture, relevant advancements in neural mathematical reasoning, tool-augmented inference, and decoding strategies are reviewed.

### *2.1 Neural Mathematical Reasoning*

Initial attempts to solve mathematical word problems relied on fine-tuning LLMs on extensive datasets of step-by-step solutions (Chain-of-Thought). While successful on elementary benchmarks like GSM8K, these models fail on Olympiad-level tasks (e.g., MATH, AIMO) due to the necessity for multi-step theorem application and out-of-distribution combinatorial reasoning. Recent foundation models leverage Reinforcement Learning (RL) to incentivize rigorous derivation. However, purely auto-regressive generation remains bound by floating-point approximation limits and attention degradation over long contexts.

### *2.2 Tool-Augmented Language Models*

To circumvent inherent arithmetic limitations, the Program of Thoughts (PoT) paradigm was introduced, instructing models to generate executable code rather than natural language answers (Chen et al., 2023). Subsequent iterations, such as ToRA (Gou et al., 2024) and PAL, demonstrated that interleaving reasoning with code execution drastically reduces hallucination rates. However, most existing implementations utilize stateless execution blocks (e.g., isolated `exec()` functions), which destroy the computational context between turns, forcing the LLM to repetitively redeclare variables and logic.

### *2.3 Uncertainty Estimation and Entropy Decoding*

Quantifying confidence in LLMs is critical for autonomous agents. Traditional methods rely on verbalized confidence scores, which are notoriously miscalibrated. Modern approaches analyze the logit distribution directly. Kuhn et al. (2024) established that semantic uncertainty can be modeled through the dispersion of token probabilities. Building upon this, entropy-based decoding strategies seek to penalize generation paths that exhibit high token-level Shannon entropy, an indicator of algorithmic guessing or indecision.

## 3. SYSTEM ARCHITECTURE AND IMPLEMENTATION

The proposed framework integrates a high-throughput Large Language Model (LLM) serving engine, an isolated stateful programmatic environment, and an entropy-aware consensus pipeline. The architecture is explicitly designed to operate within heavily constrained hardware environments—characterized by a strict 9-hour limit and finite Video RAM (VRAM) without internet access.

### *3.1 Hardware Modeling and High-Throughput Inference*

In offline evaluation environments, computational efficiency is paramount. To bypass disk I/O bottlenecks during system initialization, model weights are aggressively preloaded into the Operating System's page cache utilizing parallel thread pooling. A local vLLM server is deployed, configured with 8-bit floating-point precision (`fp8_e4m3`) for the Key-Value (KV) cache.

The necessity of quantization is demonstrated by modeling the VRAM constraints. The memory required for the KV cache $M_{KV}$ scales linearly with the batch size $B$, context length $L$, hidden dimension $D$, and the number of layers $N_{layers}$:

$$M_{KV} = 2 \cdot B \cdot L \cdot D \cdot N_{layers} \cdot P_{bytes} \quad (1)$$

By reducing the precision $P_{bytes}$ from 16-bit (2 bytes) to `fp8_e4m3` (1 byte), the memory footprint of the attention mechanism is halved. Furthermore, Prefix Caching is implemented to separate the shared system prompt memory ($M_{shared}$) from the unique generation trajectories ($M_{unique}$). The total memory dynamically allocates as:

$$M_{total} = M_{shared}(L_{prompt}) + \sum_{i=1}^{B} M_{unique}(L_{gen,i}) \quad (2)$$

This memory optimization enables the system to support a maximum context length of $65,536$ tokens across 16 concurrent sequences without out-of-memory (OOM) failures.

Table 1. System Hyperparameters and Constraints

| Parameter | Value | Description |
|---|---|---|
| Max Context | $65,536$ | Maximum sequence length |
| Worker Threads | 16 | Concurrent execution paths |
| Sampling Temp | 1.0 | Enables diverse exploration |
| Min-p | 0.02 | Truncates low-probability tail |
| Precision | `fp8_e4m3` | Reduces memory footprint |
| Sandbox Timeout | 6 s | Prevents infinite code loops |
| Top Logprobs ($k$) | 5 | Sampled for entropy calc |

### *3.2 Jupyter IOPub Message Parsing and Stateful Execution*

Unlike traditional string-based `exec()` implementations, the proposed system employs a genuine `KernelManager` instance acting over ZeroMQ sockets. This allows the system to capture real-time streaming output and cleanly segregate `stdout` from `stderr`.

The communication relies on parsing the `IOPub` channel. When the LLM generates a block of code, it is transmitted to the kernel. The system continuously polls the `IOPub` socket with a 1.0 second timeout, aggregating messages until an `idle` status is broadcast. The system categorizes messages by their `msg_type`:

- `stream`: Captures standard `print()` outputs, routing them to the model's observation space.
- `error`: Captures Python tracebacks, formatting them by stripping ANSI escape sequences before returning them to the LLM for autonomous debugging.
- `execute_result`: Captures the implicit return values of the final evaluated expression.

This robust parsing allows the LLM to write modular code over multiple conversational turns, storing massive arrays or symbolic expressions in the kernel's RAM rather than the LLM's context window. To prevent floating-point inaccuracies from compounding during iterative calculations, the precision environment is explicitly forced to 64 decimals by injecting `mpmath.mp.dps = 64` during the kernel initialization phase.

### *3.3 Autonomous Error Recovery via Finite State Machine*

A critical vulnerability of Code-LLMs is syntax failure. The architecture implements an autonomous error-recovery feedback loop, modeled as a deterministic Finite State Machine (FSM).

Let $S_0$ be the *Generation State*, where the LLM writes code. If the `IOPub` channel returns an `error` message, the system transitions to $S_1$, the *Diagnostic State*. The raw `stderr` is appended to the context window as a `ToolResponse`. The agent is strictly prompted to analyze the traceback and correct the flaw, transitioning back to $S_0$. If the system loops between $S_0$ and $S_1$ for more than 5 consecutive turns without yielding a valid output, it transitions to a *Termination State* ($S_F$) to conserve compute budget, discarding that specific generation branch.

### *3.4 Dynamic Resource Budgeting*

Time management is critical in competitive AI frameworks. With an absolute notebook limit of $T_{limit} = 17,400$ seconds and $N = 50$ problems, a static timeout allocation often results in catastrophic failure on later problems. The system implements a dynamic budget $B_t$ for each problem:

$$B_t = \min\left(\max(T_{left} - R, T_{base}), T_{high}\right) \tag{3}$$

Where $T_{left}$ is the remaining global time, $R = (N_{remaining} - 1) \times T_{base}$ is the reserved time to ensure all future problems receive at least a baseline attempt, $T_{base} = 300$ seconds, and $T_{high} = 900$ seconds. This allows the system to spend up to 15 minutes on highly complex problems early in the run, gradually becoming more aggressive with early-stopping as global time diminishes.

## 4. PROMPT ARCHITECTURE AND PERSONA DESIGN

To prevent the multi-agent generation paths from prematurely converging on identical flawed logic (consensus collapse), semantic diversity is induced at the system level through explicit prompt engineering.

The inference pool distributes specific mathematical personas across the 16 concurrent threads. This is achieved by rotating the system prompts. The baseline prompt enforces a strict reasoning protocol:

Listing 1. Baseline IMO-Expert System Prompt

```
You are an elite mathematical problem solver with
    expertise at the International Mathematical
    Olympiad (IMO) level. Your goal is to find the
    correct answer through rigorous mathematical
    reasoning.

# Problem-Solving Approach:
1. UNDERSTAND: Carefully read and rephrase the problem.
2. EXPLORE: Consider multiple solution strategies.
3. PLAN: Select the most promising approach.
4. EXECUTE: Work through your solution methodically.
5. VERIFY: Check your answer by substituting back.

# Verification Requirements:
- Cross-check arithmetic manipulations
- Ensure dimensional consistency
- The final answer must be a non-negative integer.
Place your final numerical answer inside \boxed{}.
```

To induce diversity, subsequent threads append a **Critical Protocol** constraint, effectively overriding the baseline behavior to force computational offloading:

Listing 2. Python/SymPy Specialist Override Protocol

```
CRITICAL PROTOCOL:
1. THINK FIRST before coding.
2. USE TOOLS to verify. Focus on programmatic
    verification.
3. The final answer must come from printed code output.
4. Return ONLY \boxed{answer}.
```

Additionally, the LLM is provided with a rigorous *Preference Prompt* injecting domain knowledge about available libraries. It explicitly advises the model to use `sympy` for exact symbolic algebra, `numpy` for large-scale numerical matrix evaluation, and `mpmath` for high-precision bounds checking. The instructions format is strictly enforced via the `openai_harmony` library, rendering high-fidelity tool-call tokens.

## 5. ENTROPY-WEIGHTED CONSENSUS PIPELINE

To derive the final definitive answer from the pool of generated candidates, an Inverse-Entropy Voting mechanism is proposed. The foundation of this approach relies on the principle that the uncertainty of the neural network can be quantified by analyzing the sharpness of the probability distribution over the vocabulary.

For each generated token sequence $S = (t_1, t_2, \ldots, t_n)$, the local vLLM server returns the top-$k$ log-probabilities (where $k = 5$). Let $P(t_{i,j})$ denote the probability of the $j$-th most likely token at generation step $i$. The token-level Shannon entropy is computed as:

$$H(t_i) = -\sum_{j=1}^{k} P(t_{i,j}) \log_2 P(t_{i,j}) \tag{4}$$

A low entropy value indicates a "peaked" distribution (high statistical confidence, common in exact arithmetic operations or code transcriptions). A high entropy value indicates a "flat" distribution, reflecting uncertainty and potential hallucination. The mean entropy for a complete generation path $P$ containing $N$ tokens is defined as the arithmetic average:

$$H_{mean}(P) = \frac{1}{N}\sum_{i=1}^{N} H(t_i) \quad (5)$$

The final answer selection follows a rigorous three-phase hierarchy, formalized in Algorithm 1.

**Algorithm 1** Entropy-Weighted Consensus Algorithm

**Require:** Set of valid answers $\mathcal{A} = \{a_1, a_2, \ldots, a_m\}$
**Require:** Mean entropies $\mathcal{H} = \{H_1, H_2, \ldots, H_m\}$
**Require:** LLM Evaluator function $Verify(problem, answer)$
**Phase 1: Unanimous Acceptance**
**if** $\exists a_i \in \mathcal{A}$ with $Count(a_i) \geq 4$ **then**
  **return** $a_i$ {Bypass further computation}
**end if**
**Phase 2: Actor-Critic Verification**
$Candidates \leftarrow \{a \in \mathcal{A} \mid Count(a) \geq 2\}$
Sort $Candidates$ in ascending order of average $H$
**for** $a_c \in Candidates$ **do**
  **if** $Verify(problem, a_c) == True$ **then**
    **return** $a_c$ {Zero-temperature verification}
  **end if**
**end for**
**Phase 3: Inverse-Entropy Fallback**
Initialize $Score(a) = 0$ for all unique $a \in \mathcal{A}$
**for** each generated path $p_k$ **do**
  $W_k \leftarrow \frac{1}{\max(H_k, 10^{-9})}$
  $Score(a_k) \leftarrow Score(a_k) + W_k$
**end for**
**return** $\arg\max_a Score(a)$

Phase 1 provides a low-cost early computation exit if strict unanimity is reached (Dynamic Early Stopping). Phase 2 operates as an Actor-Critic mechanism, leveraging the LLM at zero-temperature ($T = 0$) to verify contested candidates. If formal verification fails, Phase 3 acts as the final deterministic fallback, aggregating votes proportionally to the inverse of their mathematical entropy.

## 6. RESULTS AND EVALUATION

To validate the efficacy of the proposed architecture, the system was evaluated on the AIMO 3 benchmark dataset. The evaluation environment mirrored strict offline constraints: dual GPU accelerators with restricted VRAM capacity and a hard global execution limit of 9 hours for 50 problems. The submitted 39-point architecture (Hernández Barreto et al., 2026b) represents the optimal configuration discovered after evaluating multiple programmatic sandboxing paradigms.

### *6.1 Comparative Performance*

The framework was benchmarked against classical test-time compute paradigms. The baseline model utilized standard auto-regressive generation coupled with Program of Thoughts and standard Majority Voting, without stateful persistence or entropy-weighted arbitration.

As detailed in Table 2, the proposed architecture achieved a final score of 39 out of 50 points (placing in the Top 2000 tier globally). The integration of the stateful Jupyter environment drastically reduced execution errors by allowing the model to iteratively correct syntax within the same contextual state. The Inverse-Entropy Voting mechanism resolved consensus ties and overruled high-frequency hallucinations in 14% of the solved problems.

Table 2. Comparative Evaluation on AIMO 3 Benchmark

| Configuration | Accuracy | Avg. Time / Prob. |
|---|---|---|
| Baseline (PoT + Maj. Vote) | 24/50 | 410 s |
| + Stateful Sandbox | 32/50 | 345 s |
| + Semantic Diversity | 35/50 | 350 s |
| **Full Pipeline (Entropy)** | **39/50** | **290 s** |

### *6.2 Performance by Mathematical Domain*

To further understand the architecture's strengths, a post-hoc analysis of the solved problems was conducted across core mathematical domains. The results, presented in Table 3, indicate that the system excels in Number Theory and Algebra, where programmatic brute-force and `sympy` algebraic simplifications are highly effective.

Table 3. Performance Distribution by Math Domain

| Domain | Solved / Total | Success Rate |
|---|---|---|
| Algebra | 14 / 15 | 93.3% |
| Number Theory | 12 / 13 | 92.3% |
| Combinatorics | 8 / 12 | 66.6% |
| Geometry | 5 / 10 | 50.0% |

Geometry remains the most challenging domain, as translating spatial relationships into pure Python code introduces translation overhead that the LLM occasionally fails to optimize.

### *6.3 Computational Efficiency and Early Stopping*

A critical advantage of the multi-phase consensus algorithm is computational budgeting. Phase 1 (Unanimous Acceptance) successfully triggered an early exit in approximately 45% of the easiest benchmark problems. By terminating redundant worker threads once a strict consensus threshold was reached, the system conserved substantial execution time. This saved compute budget was autonomously reallocated to structurally complex problems later in the dataset.

## 7. ABLATION STUDIES

To rigorously justify the hyperparameters and architectural paradigms selected for the final submission, several ablation experiments were conducted on a localized validation set mimicking AIMO constraints.

### *7.1 Impact of Min-P Sampling*

Standard Nucleus Sampling (`top_p`) often struggles in mathematical contexts because it truncates the vocabulary purely based on cumulative probability, occasionally cutting off necessary operators. The implementation utilizes `min_p = 0.02`, which scales the probability threshold dynamically based on the most likely token. Table 4 demonstrates that `min_p` yields higher structural integrity in generated code compared to `top_p`.

Table 4. Sampling Strategy Ablation (Validation Set)

| Sampling Method | Syntax Error Rate | Accuracy |
|---|---|---|
| Top-K ($k = 40$) | 18.2% | 28/50 |
| Top-P ($p = 0.9$) | 12.5% | 32/50 |
| **Min-P ($p = 0.02$)** | **4.1%** | **39/50** |

### *7.2 Impact of the Stateful Sandbox*

When the stateful `jupyter_client` persistence was replaced with isolated, stateless `exec()` blocks, the system suffered a 20% drop in accuracy. Stateless execution forced the LLM to repeatedly import heavy libraries (`sympy`, `numpy`) and redefine complex custom functions for every verification step. This not only consumed valuable context window tokens but also increased the probability of syntax hallucination during repetitive code generation.

### *7.3 Comparative Analysis: Pure Python vs. Hybrid C++ Execution*

To further understand the optimal environment for programmatic LLM verification, an alternative architecture was developed and evaluated (Hernández Barreto et al., 2026a). This secondary implementation integrated a hybrid execution sandbox, equipping the Jupyter kernel with a native C++ compiler (`g++ -O3 -std=c++17`). The LLM was explicitly prompted to utilize Python for symbolic manipulation, but to deploy multi-line C++ scripts for massive combinatorial brute-forcing (exceeding $10^8$ iterations).

Additionally, this variant implemented a network-fault tolerance protocol (Exponential Backoff for End-of-Stream errors) and a high-divergence early abort mechanism, formalized in Algorithm 2.

**Algorithm 2** Adaptive Generation and Fault Tolerance

**Require:** Max Retries $M = 3$, Backoff array $\mathcal{B} = [1.0, 2.0, 4.0]$
**Require:** Attempt Threshold $T = 6$, Max Frequency Limit $F = 3$
**Part 1: EOS Recovery Loop (Per Turn)**
**for** $r = 0$ to $M$ **do**
  $stream \leftarrow$ Execute LLM Completion
  **if** *stream* throws End-Of-Stream Exception **then**
    Sleep for $\mathcal{B}[r]$ seconds
    Continue to next retry
  **end if**
  Return generated tokens
**end for**
**Part 2: Divergence Abort (Global Monitor)**
Let $\mathcal{R}$ be the list of current generated answers
**if** $Length(\mathcal{R}) \geq T$ **then**
  $f_{max} \leftarrow$ Frequency of most common answer in $\mathcal{R}$
  **if** $f_{max} \leq F$ **then**
    Halt all worker threads (High divergence detected)
    Return valid answers generated thus far
  **end if**
**end if**

Despite the theoretical advantage of execution speed provided by compiled C++, this hybrid architecture achieved a lower score of 37 points, compared to the 39 points achieved by the pure-Python implementation. The comparative metrics, detailed in Table 5, isolate the primary vectors of degradation.

Table 5. Architectural Comparison: Pure Python vs. Hybrid C++

| Metric | Pure Python | Hybrid C++ |
|---|---|---|
| AIMO 3 Score | 39 / 50 | 37 / 50 |
| Phase 2 (Actor-Critic) | Active | Disabled |
| Code Syntax Error Rate | 4.1% | 14.7% |
| Avg. Turn Latency | 18.2 s | 24.5 s |
| Timeouts (TLE) | 0 | 3 |

Empirical evidence indicates two primary causes for the performance drop in the hybrid model. First, LLMs exhibit a significantly higher hallucination rate when writing complex C++ templates and memory-managed code compared to Python syntax, driving the Syntax Error Rate up to 14.7%. Furthermore, the compilation overhead and strict memory limits imposed during the execution of compiled binaries occasionally triggered Time Limit Exceeded (TLE) faults within the sandbox. Second, the 37-point architecture omitted the Phase 2 Actor-Critic verification to save computational budget, relying entirely on Phase 3 Inverse-Entropy fallback. This omission confirms that zero-temperature verification remains a critical stabilizing component in the overall consensus pipeline.

## 8. CONCLUSION

This paper presented an advanced, agentic inference framework designed to tackle Olympiad-level mathematical reasoning under strict offline hardware constraints. By integrating a persistent, stateful Jupyter environment, the architecture successfully decoupled abstract logical planning from rigorous numerical execution, mitigating the compounding arithmetic hallucinations typical of large language models. Furthermore, the introduction of an Inverse-Entropy Voting mechanism provided a statistically grounded approach to consensus arbitration, proving that token-level uncertainty is a far more reliable metric than raw generation frequency.

Achieving 39 points in the AIMO 3 benchmark validates the scalability and robustness of this neuro-symbolic approach. Crucially, architectural ablation demonstrated that while compiled languages offer theoretical brute-force speed, the syntactic simplicity and native precision of Python proved superior for LLM-driven verifiable reasoning, minimizing syntax hallucination. Future work will explore the integration of this entropy-aware decoding strategy with foundation models trained explicitly via Reinforcement Learning for mathematical formalization, potentially closing the gap between informal natural language problem-solving and strict formal verification.

## REFERENCES

Brown, B., Cassano, F., Chen, X., et al. (2024). Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*.

Chen, W., Ma, X., Wang, X., and Cohen, W.W. (2023). Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*.

DeepSeek-AI (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Gou, Z., Shao, Z., Gong, Y., Shen, Y., Huang, Y., Duan, N., and Chen, W. (2024). Tora: A tool-integrated reasoning agent for mathematical problem solving. In *The Twelfth International Conference on Learning Representations*.

Hernández Barreto, A.D., Díaz Domínguez, A.E., and Morales Escobar, K.Y. (2026a). Aimo 3: Hybrid python/c++ sandbox and eos recovery (37-point submission). `https://www.kaggle.com/code/dlthhb/what-to-eat`. Accessed: April 2026.

Hernández Barreto, A.D., Díaz Domínguez, A.E., and Morales Escobar, K.Y. (2026b). Aimo 3 solve: Entropy-weighted consensus architecture (39-point submission). `https://www.kaggle.com/code/dlthhb/aimo-3-solve`. Accessed: April 2026.

Kuhn, L., Gal, Y., and Farquhar, S. (2024). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Twelfth International Conference on Learning Representations*.

Trinh, T.H., Wu, Y., Le, Q.V., He, H., and Thang, L. (2024). Solving olympiad geometry without human demonstrations. *Nature*, 625(7995), 476–482.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Dai, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.